

# Relatório técnico: o modelo “mesmo fato” do Lume

Residência em IA (UnB), challenge 01, desinformação. Entrega de 09/10/2026. Autor: Grupo 7.

## Resumo

O Lume é o app do grupo que pesquisa fontes para uma notícia. Para cada fonte que a busca traz, ele precisa responder: **esta fonte fala do mesmo fato que a notícia?** Hoje quem responde é uma regra escrita à mão (`EventFrame`), que compara quem agiu, a ação, o assunto e a data. Ela erra pouco, mas acha pouco.

Treinamos um modelo que responde a mesma pergunta. Na prova (100 notícias capturadas e lacradas antes de qualquer rótulo, olhadas uma única vez), de cada 10 fontes que são o mesmo fato:

| Decisor                                | Acha       | Do que mostra, acerta | Contra o app do MVP      |
|----------------------------------------|------------|-----------------------|--------------------------|
| Regra do MVP do grupo (MPNet original) | 3,5        | 73%                   | a base                   |
| Regra com o MPNet ajustado             | 3,6        | 75%                   | +0,1                     |
| Plano B (o modelo sem o BERTimbau)     | 7,1        | 72%                   | +3,7 [+2,7; +4,6]        |
| <b>6B, o modelo escolhido</b>          | <b>7,6</b> | <b>74%</b>            | <b>+4,1 [+3,0; +5,2]</b> |

Entre colchetes, a faixa de 95% sorteando notícias inteiras. O modelo acha mais que o dobro das confirmações e erra na mesma proporção que a regra. Pelo critério combinado antes de começar, passou.

## 0. O problema, em termos de dado

| Micro ponto       | O que é                                                                                          |
|-------------------|--------------------------------------------------------------------------------------------------|
| Cada linha        | um par: a afirmação da notícia e o título de uma fonte que a busca do app trouxe                 |
| O que o modelo vê | 13 informações que o app já tem na hora (tabela da seção 1.3) e a nota de um modelo de linguagem |
| O alvo            | <code>mesmo</code> : a fonte relata o mesmo acontecimento (sim ou não)                           |
| As classes        | desequilibradas: uns 12 de cada 100 pares são “mesmo”                                            |
| O tipo de modelo  | classificação binária; regressão logística empilhada com um BERTimbau ajustado                   |

| Micro ponto | O que é                                 |
|-------------|-----------------------------------------|
| A régua     | a regra do app, medida nos mesmos pares |

A acurácia não serve aqui: dizer “não” para tudo acerta 88% e não acha nenhuma confirmação. As medidas são as duas que o leitor do app sente (seção 3).

## 1. Preparação dos dados e treinamento

### 1.1 De onde vêm os dados

Os pares saíram do próprio app. Capturamos manchetes reais, rodamos a busca do Lume em cada uma (o diagnóstico ao vivo do app grava a busca inteira: as fontes, a decisão da regra, as datas) e rotulamos cada par.

| Rodada | Quando | Pares | Papel                                          |
|--------|--------|-------|------------------------------------------------|
| 1      | 30/09  | 401   | 14 notícias: buscas do MVP do Guilherme        |
| 2      | 06/10  | 983   | 20 notícias nossas                             |
| 3      | 07/10  | 8.910 | 200 notícias                                   |
| 4      | 08/10  | 4.298 | 100 notícias, lacradas antes da busca: a prova |

**A rotulagem** seguiu uma regra escrita, com desempates fixados. Dois juízes de linguagem (Claude Opus e um segundo modelo) rotularam sem ver um ao outro; o trecho literal da fonte era obrigatório e a conferência rejeitava o que não batesse. Onde discordaram, um terceiro (Gemini) desempatou. Uma amostra estratificada rotulada por um de nós, antes de ver qualquer juiz (a “âncora”), decidiu se o conjunto valia como gabarito. Na rodada 4, a concordância entre juízes foi  $\kappa = 0,82$  e a dos juízes com a âncora, 88,7 de 100.

### 1.2 Limpeza

As regras de limpeza:

- os pares que serviram de exemplo nas instruções dos juízes e os 22 sintéticos ficam no treino e fora de toda medida;
- a resposta da regra do app é recalculada do registro (em parte dos pares vinha a resposta de outra versão);
- marcadores de ausência (“sem data”, vazio) viram ausência explícita;
- notícias com a mesma afirmação ficam no mesmo grupo. Isso veio de uma cola real do ciclo 1: três manchetes capturadas em dois dias caíram em grupos diferentes, e 10 pares idênticos ficaram no treino e na validação.

Um problema de rótulo apareceu na exploração: **131 pares “mesmo fato” tinham a fonte com mais de um mês**, e os juízes nunca viram a data (o pacote mostrava a data do registro, quase sempre vazia). Exemplo: a queda da Starlink de hoje casada com uma de 2025. Tentamos re-julgar esses pares com a data à vista, duas vezes, com uma conferência humana combinada antes (limite: 3 discordâncias em 10).

As duas pararam na conferência. Os rótulos originais ficaram, e esses pares viraram uma fatia de apoio, medida com e sem.

### 1.3 As informações que o modelo vê

| Informação                                                                         | O que mede                                                                                                        |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>cosseno</code>                                                               | semelhança de sentido entre a afirmação e o título, pelo MPNet ajustado num dataset público de paráfrases (ASSIN) |
| <code>sobreposicao</code>                                                          | fração de palavras em comum                                                                                       |
| <code>idade_dias</code> , <code>tem_idade</code>                                   | dias entre a fonte e a notícia; se há data                                                                        |
| <code>comparou</code>                                                              | se a comparação estruturada da regra rodou na busca                                                               |
| <code>titulo_cortado</code>                                                        | se o título veio cortado (o registro corta em 110 caracteres)                                                     |
| <code>quem</code> , <code>acao</code> , <code>assunto</code> , <code>quando</code> | as partes da comparação da regra                                                                                  |
| <code>buscador</code>                                                              | Google Notícias, Bing Notícias, Bing web, Wikipédia                                                               |
| <code>tipo_afirmacao</code>                                                        | o tipo que o app deu à afirmação (acontecimento, contagem...)                                                     |
| <code>nota_texto</code>                                                            | a nota do BERTimbau ajustado, que lê a afirmação e o título juntos (só no 6B)                                     |

Só entra o que o app tem na hora de decidir. A resposta final da regra foi testada como candidata e não entrou (empate).

### 1.4 Divisão

| Monte    | Pares | Notícias       | Para quê                            |
|----------|-------|----------------|-------------------------------------|
| Estudo   | 7.920 | 182            | o modelo aprende                    |
| Simulado | 2.374 | 49 (rodada 3)  | escolher o modelo e a nota de corte |
| Prova    | 4.298 | 100 (rodada 4) | medir uma vez, no fim               |

Fontes da mesma notícia nunca ficam em dois montes. A prova ficou num cofre, com impressão digital conferida na abertura e contador de aberturas: foi aberta uma vez, na etapa 14.

### 1.5 Pré-processamento

Números padronizados (média e desvio do estudo); idade em `log(1 + dias)`, com a mediana do estudo onde falta data; categorias em colunas 0/1, com valor desconhecido somando zero. Tudo aprendido só no estudo, dentro de um `Pipeline` do scikit-learn, que é o objeto salvo em `modelo/modelo_6b.pkl`.

### 1.6 Treino

- **A régua** (o “plano B”): regressão logística,  $c = 0,1$ , classes equilibradas, 34 colunas.
- **O BERTimbau** (neuralmind/bert-base-portuguese-cased), ajustado como *cross-encoder*: lê a afirmação e o título juntos e dá uma nota de 0 a 1. Duas passadas pelo estudo, taxa  $2e-5$ , peso maior para os “mesmo”. Para a nota poder entrar na logística sem cola, cada par do estudo recebeu a nota de uma versão treinada sem a notícia dele (5 dobras por notícia); o simulado e a prova, a do modelo treinado com o estudo inteiro.
- **O 6B**: a régua com a nota do BERTimbau como mais uma informação,  $c = 0,03$  (etapa 11).

## 2. Critérios de seleção da abordagem

### 2.1 O critério veio do produto, e foi decidido antes do dado

No ciclo 1, o critério era “precisão de pelo menos 90%”. Ele veio do hábito do repositório, não do app: nem a regra do app passava nele. O modelo achou o dobro das confirmações da regra e foi reprovado por esse número. A lição virou o contrato do ciclo 2, decidido antes de qualquer dado novo:

1. **Achar mais que a regra do app**, com a faixa da diferença toda acima de zero;
2. **não errar mais que 3 pontos acima da regra**. A margem de 3 é o tamanho do acaso numa prova deste tamanho: sem margem, um modelo tão bom quanto a regra seria reprovado metade das vezes por sorte.

A nota de corte de cada candidato é a do ponto em que ele erra o mesmo que a regra no simulado; assim todos são medidos pela mesma régua.

### 2.2 Onze candidatos, a mesma régua

| Candidato                                                              | Acha de cada 10 (simulado) | Contra a régua                     |
|------------------------------------------------------------------------|----------------------------|------------------------------------|
| Régua (logística)                                                      | 7,53                       | é a base                           |
| + semelhança × idade; + resposta da regra; as duas; sem a rodada 1     | 7,37 a 7,60                | empates                            |
| Árvores em sequência (HistGradientBoosting); logística com cruzamentos | 7,53 e 7,60                | empates                            |
| CatBoost com o site                                                    | 6,97                       | perde                              |
| Só palavras (TF-IDF)                                                   | 0,93                       | perde                              |
| BERTimbau sozinho                                                      | 6,00                       | perde                              |
| App híbrido (regra onde ela compara, modelo no resto)                  | 6,83                       | perde                              |
| <b>6B: régua + nota do BERTimbau</b>                                   | <b>8,10</b>                | <b>ganha: +0,57 [+0,10; +1,12]</b> |

Empate fica com o mais simples. O 6B foi o único com a faixa toda acima de zero. Custo, aceito antes de seguir: no app, o BERTimbau (436 MB) precisa de um servidor. Por isso o plano B (a régua) vai junto, para quando o servidor não responde.

## 2.3 Afinação

48 combinações (o quanto o modelo “se segura”, o peso dos rótulos desempatados, a nota do texto como sai ou “esticada”), medidas dentro do estudo em 5 partes por notícia, sem gastar o simulado. A regra, escrita antes: só sai da posição de fábrica quem ganhar por mais de 0,1 de cada 10. Ficou  $c = 0,03$ ; no simulado, empata com a de fábrica.

## 3. Métricas e análise dos resultados

### 3.1 As medidas

| Medida                                         | Pergunta                                                   | Por quê                                       |
|------------------------------------------------|------------------------------------------------------------|-----------------------------------------------|
| Cobertura ( <i>recall</i> ), “acha de cada 10” | das fontes que são o mesmo fato, quantas o decisor aponta? | é o que o leitor ganha                        |
| Precisão                                       | do que o decisor mostra, quanto está certo?                | é o que o leitor perde quando ele erra        |
| PR-AUC                                         | a ordem das notas é boa em todos os cortes?                | apoio; o acaso fica em 0,12                   |
| Faixa por bootstrap de notícias                | a diferença é real ou sorte?                               | fontes da mesma notícia não são independentes |

### 3.2 Simulado e prova

|              | Simulado: acha | Simulado: precisão | Prova: acha | Prova: precisão |
|--------------|----------------|--------------------|-------------|-----------------|
| Regra do app | 4,13           | 85,5%              | 3,57        | 75,0%           |
| 6B           | 8,07           | 85,5%              | 7,58        | 74,3%           |

Na prova: exigência 1, +4,01 de cada 10 [+2,82; +5,16], passa; exigência 2, -0,7 ponto [-8,3; +7,4] com limite -3, passa. PR-AUC do 6B: 0,916 no simulado e 0,814 na prova.

No simulado, o pipeline tinha medido 8,10; o notebook mede 8,07. O corte foi gravado com 6 casas, e a única fonte que estava exatamente nele ficou de fora (uma em 300). A prova sempre usou o corte gravado.

A prova foi uns 10 pontos mais difícil que o simulado **para os dois**. O contrato passou porque comparava os dois no mesmo conjunto; um critério fixo de “85%” teria reprovado os dois.

### 3.3 Contra o app do MVP

A semelhança do MPNet só decide a regra num caso (quem e ação batem, as palavras do assunto não). Refizemos a regra do MVP do grupo, com o MPNet original e os cortes 0,72/0,55, nesse ramo: 56 das 4.298 fontes, e ela difere da nossa em 22 (0,5%). A reconstrução foi conferida: refazendo a nossa regra do mesmo jeito, ela bate com o app em 93% do ramo. **O ganho do app vem do modelo**; o MPNet ajustado importa como a informação que o modelo mais usa.

### 3.4 Onde o modelo erra

| Idade da fonte (prova) | O 6B acerta, do que mostra |
|------------------------|----------------------------|
| Mesmo dia ou véspera   | 85 de cada 100             |
| 2 a 30 dias            | 55 de cada 100             |
| Mais de um mês         | 25 de cada 100             |

O erro tem endereço: fonte do mesmo assunto publicada dias antes. Os piores enganos no simulado são o mesmo assunto contado por outro recorte (“quem se elegeu” contra “quem **não** se elegeu”), e parte deles parece rótulo errado. Por isso, no app, o selo “provável mesmo fato” só aparece sozinho para fonte do mesmo dia ou da véspera; nas outras, o aviso é “mesmo assunto, confira a data” (no simulado, 8,9 e 5,2 acertos de cada 10).

A nota do modelo não é uma “chance”: quando ele dá 0,75, só 1 em 4 é o mesmo fato; quando dá 0,97, quase 9 em 10. Ela não vai para a tela como porcentagem sem uma calibração.

### 3.5 Testes de vazamento

- Alvo embaralhado: a PR-AUC cai para 0,11 a 0,28 (acaso 0,126), contra 0,902 do modelo de verdade.
- Nenhum par repetido entre estudo e simulado.
- Um canário (dado sintético em que nenhum modelo honesto passa de 0,80) deu 0,689 e 0,731.

## 4. Principais desafios

---

1. **Definir “mesmo fato”.** Notícias de recorrência (“outra chegada de remédio”, “outro corpo identificado”) e fonte velha sem data confundem juízes e gente. A regra de rotulagem foi escrita e reescrita; a âncora humana decidiu.
2. **A ordem das opções puxa o clique.** Na rodada 3, a âncora humana caiu de gabarito para “prata” porque a resposta provável estava sempre em cima. A partir daí, a ordem foi sorteada por par.
3. **Um critério herdado reprovou o ciclo 1** (seção 2.1).
4. **Vazamento por grupo** (seção 1.2), achado pelo teste de repetição.
5. **A prova foi mais difícil que o simulado** para todos os decisores.
6. **Engenharia no Windows:** uma biblioteca do pipeline corrompia arquivos por codificação; a conversão automática de fim de linha do git quebrava a conferência de hash; no pandas 3, 625 fontes sem data sumiam caladas de um recorte.
7. **Levar o modelo ao app:** o modelo só vale com as informações calculadas exatamente como no treino (título cortado em 110 caracteres, a primeira comparação da regra, o dia de Brasília). E uma afirmação nossa estava errada e foi corrigida: o plano B não roda sozinho no celular, porque a semelhança do MPNet vem do Ollama no computador.

## 5. Aprendizados

---

- **O critério de sucesso vem do produto e é decidido antes do dado.** Medido contra uma régua no mesmo conjunto, ele aguenta a mudança de dificuldade entre simulado e prova.
- **Contar o fenômeno antes de atacá-lo.** A contagem da etapa 1 mudou o alvo do ciclo (o erro caro era fonte velha, não o que se esperava).
- **Expectativa escrita antes de rodar.** Cada script e caderno traz a faixa esperada; as que erraram estão registradas (a nota “esticada” piorou o modelo; a prova saiu 10 pontos abaixo; os piores erros não eram o que esperávamos).
- **Empate fica com o mais simples.** Dos onze candidatos, só um ganhou com segurança; árvores e CatBoost não pagaram a complexidade.
- **A prova se abre uma vez.** Escolher olhando a prova infla o número sem dar erro.
- **Um nulo bem medido é resultado.** O ciclo 1 não passou e isso gerou o contrato que fez o ciclo 2 passar.

## 6. Limitações

---

- A prova tem 100 notícias de um único dia; a faixa da precisão é larga ( $\pm 8$  pontos).
- O selo com a data foi escolhido olhando a prova e o simulado: a primeira época nova rotulada é a medida independente.
- Parte dos rótulos de fonte velha é duvidosa; a revisão às cegas é o primeiro item do próximo ciclo.
- O 6B precisa de um servidor para o BERTimbau; sem ele, o plano B acha um pouco menos.

## 7. Como reproduzir

---

Abra `avaliar_o_modelo.ipynb` de dentro desta pasta e rode tudo (Python 3.12, `pandas`, `numpy`, `scikit-learn` 1.9.1, `matplotlib`). Ele carrega o modelo, refaz as medidas do simulado e da prova e refaz o treino da logística. Os pesos do BERTimbau (436 MB) vão separados; com eles, uma célula opcional recalcula a nota do texto. O caminho das 16 etapas está em `cadernos-do-pipeline/` e `figuras/`, e o diário de cada etapa, no repositório da residência (`ml_pipeline/PIPELINE.md`).